

Deep Architectures

תזכורות: 1. ראינו בעיות עם MLP¹:

- קשה לעשות אופטימיזציה כי יש המון פרמטרים.
- זה גם גורם ל-overfitting.

במקרה של תמונות, ראינו שאפשר לשפר עם CNN². ההבדל העיקרי היה שבכל אחת מהשכבות יש parameter sharing - כל יחידה חולקת את הפרמטרים שלה עם יחידות אחרות, וה-output שונה כי הן מסתכלים על אזורים שונים ב-input.

באמצעות parameter sharing:

- אנחנו חוסכים בכמות הפרמטרים.
- אנחנו פחות רגישים להזזות של הקלט.

2. את הרשתות האלו אימנו באמצעות gradient descent. ראינו שבגלל שהרשת כל-כך כבדה, צריך לבצע את חישובי הגרדיאנט ביעילות.

3. יש שלושה מקורות לשגיאות:

- approximation error $f_H \neq f$ - מרחב ההיפוטזות שלנו H לא מספיק אקספרסיבי בשביל לייצג נאמנה את הפונקציה האמיתית f .
- estimation error $f_{H,S} \neq f_H$ - אנחנו לומדים מתוך מדגם סופי S . יש בעיה של הכללה.
- optimization error $f_{H,S} \neq f_{H,S}^*$ - לא בטוח שנוכל למצוא את הפונקציה שמביאה את ה-loss למינימום.

במקרה של MLP יש בעיה גדולה של estimation error. CNN מקטין את הבעיה הזאת כי הוא מקטין את מספר הפרמטרים - אבל זה עובד רק למקרים מסויימים בהן יש לוקאליות (למשל תמונות)

כשלמדנו את הפיצ'רים האלה מתוך data, לא באמת כיווננו אותם. לא רמזנו להם מה הם צריכים לחפש. אם איתחנו שני פילטרים בצורה זוהה, צעדי העדכון יהיו זהים והם כנראה יהיו זהים (יצביעו על אותו פיצ'ר)

אפשר להסתכל על הפילטר עצמו בתור תמונה - לנרמל אותם לתחום $[0, 256]$, ואז להציג אותם בתור תמונות. כשעושים את זה רואים שיש כמה פילטרים שבודקים אותו דבר - אבל בסיבובים שונים. בניסויים ניורולוגים רואים שככה גם מוחות אמיתיים (של יונקים, לפחות) עובדים - ניורונים ספציפיים מגיבים לזוויות ספציפיות.

איך אפשר, מתוך תאים פשוטים, לייצר קומבינציות שמזהות תבניות מורכבות? יש למשל תאים שהרגישות שלהם היא נקודה כאה עם אזור בהיר מסביב. אפשר לבנות את זה עם פילטר פשוט ב-CNN - משקלות שליליות באמצע וחיוניות מסביב. אם מחברים כמה כאלה בשורה נקבל רגישות לאוריינטציה מסויימת.

¹Multi-Layer Perceptrons
²Convolutional Neural Network

שיפורים לאופטימיזציה והכללה

- הבעיה בסיגמואיד היא שאם אנחנו קרובים לאפס הרגדיאנט יביא להתכנסות - אבל אם אנחנו רחוקים מהאפס זה בעיה.
- ב RLU יש בעיה דומה.

אנחנו רוצים לאתחל את המשקלות כך שהן יהיו קרובות לאפס. אבל אנחנו רוצים גם שהקלט יהיה מאוזן. בשביל זה נרצה לעשות עיבוד מקדים לקלט. צעד אחד שמקובל לעשות הוא להזיז את הנתונים ככה שהממוצע יהיה קרוב לאפס:

$$x_i \leftarrow x_i - \mathbb{E}[x_i] \quad \forall_i$$

נרצה גם לדאוג שהסקאלה של פיצ'רים³ שונים תהיה באותו סדר גודל - ולכן נחלק בסטיית התקן:

$$x_i \leftarrow \frac{x_i}{\text{std}(x_i)}$$

גם אחרי הדברים האלה עדיין יכולה להיות קורולציה בין הפיצ'רים, ויש פיתוי לנרמל גם את זה (להפוך את covariance למטריצת היחידה), אבל לא כדאי לעשות את זה כי:

- יכול להיות שהתלויות הסטטיסטיות האלו חשובות, וכן נרצה שהמודל ילמד אותן.
- הערה: בכל מקרה כל הנורמליזציות האלו הן לינאריות - הרשת הייתה יכולה ללמוד אותן.
- הסיבוב בעייתי - הוא דורש להפוך מטריצות גדולות ורועשות. זה עלול לעשות overfitting.

BatchNorm

לעשות את הנורמליזציה על כל ה-batch כבד - ולכן נרצה אולי לעשות BatchNorm - לחשב את הממוצע והשונות על חלק מהדגימות $x_1, \dots, x_N \in \mathbb{R}^D$. ואז נחשב per-channel את הממוצע $\mu_j = \frac{1}{N} \sum_{i=1}^N x_{i,j}$ ואת השונות $\sigma_j^2 = \frac{1}{N} \sum_{i=1}^N (x_{i,j} - \mu_j)^2$ ומנרמלים

$$\hat{x}_{i,j} = \frac{x_{i,j} - \mu_j}{\sqrt{\sigma_j^2 + \epsilon}}$$

(את ה- ϵ מוסיפים כדי למנוע חלוקה באפס וגם כדי למנוע בעיות נומריות כשהשונות מאוד קטנה)

אפשר לעשות את הנרמול בכל שכבה - אבל נשים לב שאת הנרמול עושים לפני האי-לינאריות.

את הנרמול אפשר לעשות גם בתור למידה:

$$y_{i,j} = \gamma_j \hat{x}_{i,j} + \beta_j$$

³ כאן מדובר על פיצ'רים של הקלט, לא של השכבות הפנימיות (למרות שגם להן אפשר לעשות נורמליזציה)

למידת מכונה
89-2511-02

מקליד: עידן אריה
מרצה: פרופ' גל צ'צ'יק
תאריך: 2026-06-01

אבל צריך לשים לב שבtest משתמשים בפרמטרים מהאימון.
תוצר לוואי של BatchNorm היא שכל דגימה מקבלת נורמליזציה שונה כאשר היא משמשת בbatchים שונים - מה שמגדיל את מספר הדגימות.

ארכיטקטורת ResNet (Residual Network)

ה deep learning מתאפיין במספר השכבות - אבל עד לפני כעשות התחום היה תקוע בסביבות 10-15 שכבות מכיוון שרשתות יותר עמוקות ביצעו גרוע - היו להן יותר פרמטרים ולכן בעיית הכללה.

אם כך, היינו מצפים לראות את הבעיה רק ב test ולא ב training. אבל ניסויים הראו שרשתות עמוקות יותר עובדות פחות טוב גם ב training - מה שאומר שזו בעיית אופטימיזציה ולא בעיית הכללה.

הבעיה היא שגרדיאנט צריך לחלחל דרך המון שכבות שמאותחלות אקראית. הגרדיאנטים בשכבות המוקדמות או שהן דואכים לאפס או שהם מתפוצצים - ולכן קשה לאמן את השכבות האלו.

ב 2012 AlexNet שיפרה את הביצועים והגיע ל 8 שכבות.
ב 2015 ארכיטקטורת ResNet הצליחה להגדיל משמעותית את העומק (152 שכבות) ולעקוף את הביצועים של בן אדם בתחרות זיהוי תמונות.
הטריק: ברשת רגילה כל שכבה מסתכלת על השכבה הקודמת. ב ResNets יש שני מרכיבים:

1. datan כמו שהוא, בלי שמשנים אותו.

2. מרכיב של שינוי אדיטיבי על datan.

הרשת לומדת רק את השינוי האדיטיבי:

$$H(x) = F(x) + x$$

במקום ללמוד את $H(x)$, לומדים רק את $F(x)$. ה loss עובר כאן בהרבה מסלולים, וזה משפר את הגרדיאנט.

את skip connection הזה עושים בחלק מהשכבות - מה שנותן לנו משפחה שלמה של מודלים (לפי איפה שמפעילים את skip connection)

אתחול המשקלות

אם מאתחלים את המשקלות גדולות או קטנות מדי מקבלים בעיות נומריות. רוצים לאתחל אותן ככה שמה שנכנס לאזור האי-לינאריות יהיה באזור האפס.
הרעיון אומר שצריך לאתחל לפי עקרון fan-in - רוצים שהשונות של הקלט לניורון תהיה קבועה.

$$\text{Var}(z) = \text{Var}\left(\sum_j w_j x_j\right) = \sum_j \text{Var}(w_j) \text{Var}(x_j) = n_{\text{in}} \text{Var}(w) \text{Var}(x)$$

רוצים $n_{\text{in}} \text{Var}(w) = 1 \iff \text{Var}(Z_1^{[1]}) = \text{Var}(x)$
בפועל אי אפשר לנרמל לגמרי כי Variance תלוי בקלט. בפועל עובד לדגום לפי:

$$W \sim N\left(0, \frac{2}{n_{\text{in}} + n_{\text{out}}}\right) \quad W \sim U\left[-\frac{\sqrt{6}}{\sqrt{n_j + n_{j+1}}}, \frac{\sqrt{6}}{\sqrt{n_j + n_{j+1}}}\right]$$

Data Augmentation

בשלב מסויים המונח regularization התחיל לשמש לכל טכניקה שמשפרת את הקלט. לפעמים אפשר להתמודד עם רגישויות ברמת הארכיטקטורה (למשל רגישות להזזות ב-CNN) אבל הרבה פעמים יותר פשוט לעשות שינויים ב-data - מה שגם עוזר להגדיל אותו (למשל לסובב, או לשנות צבע)

MixUp

אפשר לעשות ממוצע משוקלל לתמונות:

$$\tilde{x} = \lambda x_i + (1 - \lambda) x_j \quad \tilde{y} = \lambda y_i + (1 - \lambda) y_j$$

כאשר: x_i, x_j ווקטורי קלט גולמיים
 y_i, y_j ווקטורי one-hot שמקודדים את התוויות

הרעיון הוא להוסיף לכל קלט משהו מהמחלקות האחרות.
איך בוחרים λ בפועל? באמצעות התפלגות $\lambda \sim \text{Beta}(\alpha, \alpha)$.

הכללה בלמידה עמוקה

בניגוד למה שחשבנו בלמידה "קלאסית", בה ברגע שמגיעים לרמה מסויימת של מורכבות, בלמידה עמוקה אחרי שממשיכים להגדיל מורכבות מגיעים למצב שהמודל ממשיך להשתפר (להקטין את השגיאה על test). למה זה עובד?

בניסוי שנעשה עם תוויות אקראיות, ראו שהרשת מקטינה את השגיאה על train setn - כלומר עושה overfitting. למה זה מפתיע? כי עד אז ציפו שיהיה overfitting כאשר יש הרבה פרמטרים, וכאן ראו שלא היה (או לפחות - היה הרבה פחות overfitting כאשר התוויות היו אמיתיות). למרות שהמודל יכול לעשות overfitting - אם יש חוקיות בdata הוא לא עושה את זה ולומד את הכלל האמיתי.

למה שהמודל יעדיף כלל שהגיוני עבור אנשים כשהוא יכול פשוט לעשות overfitting? נראה שמודלים שמאוד עשירים מתחילים ללמוד את המבנים הגדולים בdata ועושים overfitting רק אם מכריחים אותם. למה זה קורה?

- אימון באמצעות SGD טוב להכללות.
 - רשתות לומדות את החוקים הטובים קודם
 - רשת מאותחלת אקראית תכיל בד"כ תת-רשת שנותנת ביצועים טובים
- זה נקרא Lottery Ticket Hypothesis - אם לוקחים אחוז מסויים מהפרמטרים, שאחרי למידה קיבלו את המשקלים הכי קטנים בערך מוחלט, עושים להם pruning, ואז מאמנים הרשת מהתחלה כאשר שאר הפרמטרים מקבלים את הערכים המקוריים שלהם - מקבלים את אותם ביצועים.